

AI-powered bioinformatic analysis using R, #5

Data Visualization I — Base R & ggplot2 Fundamentals

Wei-Hua Chen (CC BY-NC 4.0)

HUST, China

2026-07-17

Table of contents I

- 1 R Base Graphics
- 2 Introduction to ggplot2
- 3 1. Layers (geoms)
- 4 2. Scales
- 5 3. Coordinate Systems
- 6 4. Facets
- 7 Summary

Section 1

R Base Graphics

Why Visualization Matters in Bioinformatics

- **Explore data:** detect patterns, outliers, clusters
- **Communicate results:** publication-quality figures
- **Validate analyses:** PCA plots, volcano plots, heatmaps

R Base Plot: Scatter Plot

Let's first see the data:

```
head(swiss)
```

	Fertility	Agriculture	Examination	Education	Catholic
Courtelary	80.2	17.0	15	12	9.96
Delemont	83.1	45.1	6	9	84.84
Franches-Mnt	92.5	39.7	5	5	93.40
Moutier	85.8	36.5	12	7	33.77
Neuveville	76.9	43.5	17	15	5.16
Porrentruy	76.1	35.3	9	7	90.57

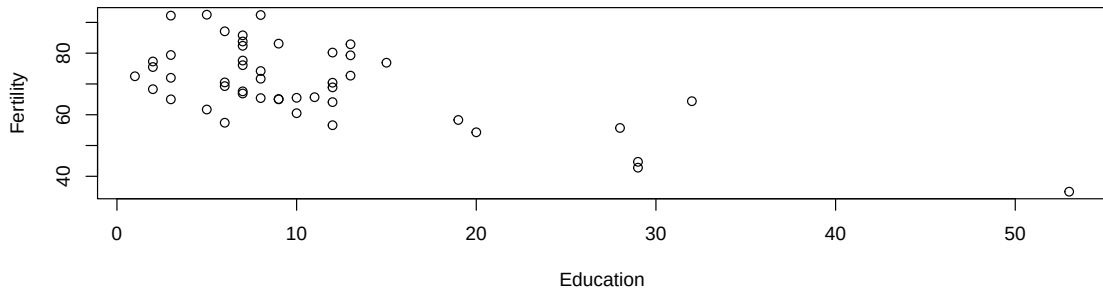
	Infant.Mortality
Courtelary	22.2
Delemont	22.2
Franches-Mnt	20.2
Moutier	20.3
Neuveville	20.6
Porrentruy	26.6



Scatter Plot with plot()

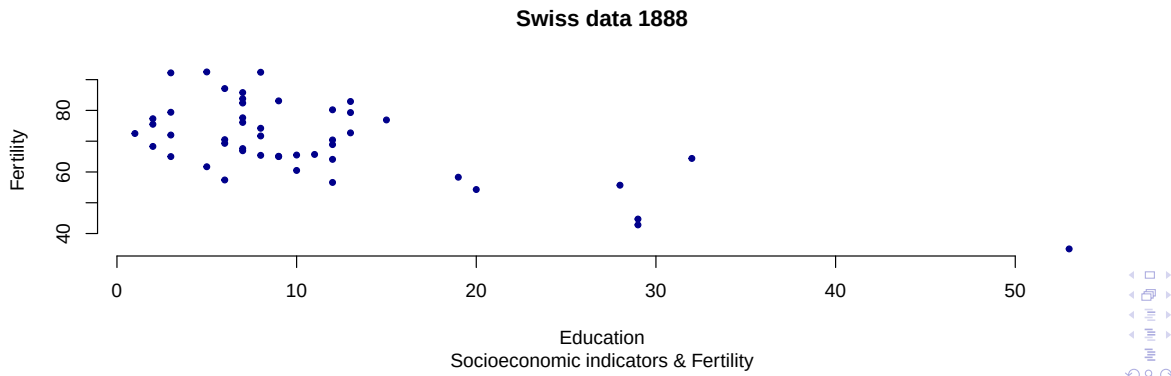
Make a scatter plot of Fertility vs. Education, note the use of with():

```
with(swiss, plot(Education, Fertility))
```



plot() Parameters

```
with(swiss, plot(Education, Fertility,  
  type = "p", main = "Swiss data 1888",  
  sub = "Socioeconomic indicators & Fertility",  
  xlab = "Education", ylab = "Fertility", col = "darkblue",  
  xlim = range(Education), ylim = range(Fertility),  
  pch = 20, frame.plot = FALSE))
```



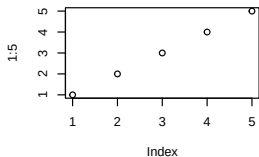
plot() Parameter Summary

- **type**: "p" points, "l" lines, "o" overplotted, "b" both, "c", "s", "S", "h"
- **main**, **sub**, **xlab**, **ylab**: labels
- **col**: color
- **xlim**, **ylim**: axis limits
- **pch**: point character (0–25, 32–127)
- **log**: "x", "y", "xy" for log-scale axes

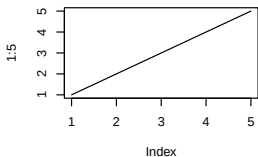
plot Types

```
par(mfrow = c(2, 4))
opts <- c("p", "l", "o", "b", "c", "s", "S", "h")
for (o in opts) {
  plot(1:5, type = o, main = paste("type =", o))
}
```

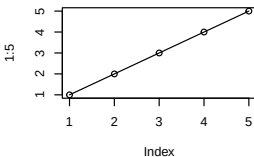
type = p



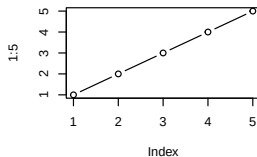
type = l



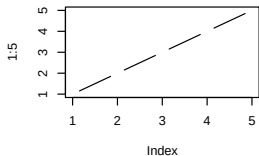
type = o



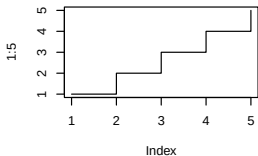
type = b



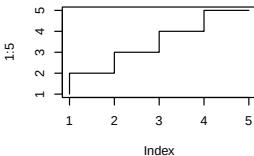
type = c



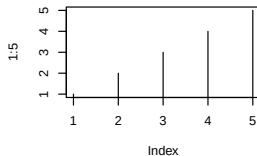
type = s



type = S



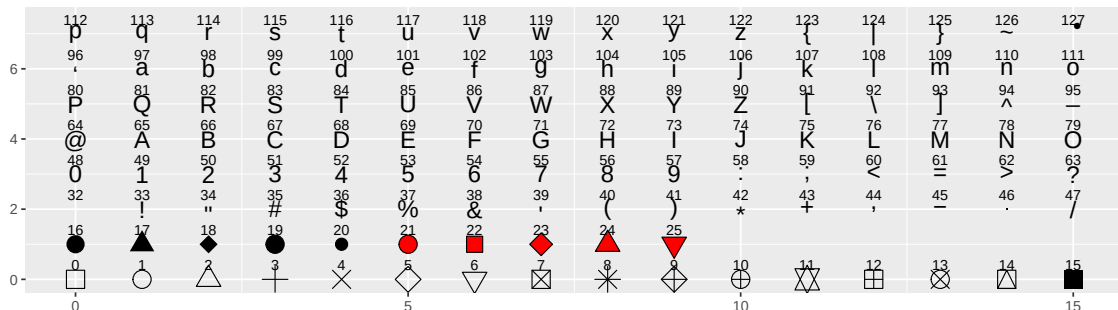
type = h



pch: Point Characters

Note: pch 21–25 have both colour (stroke) and fill.

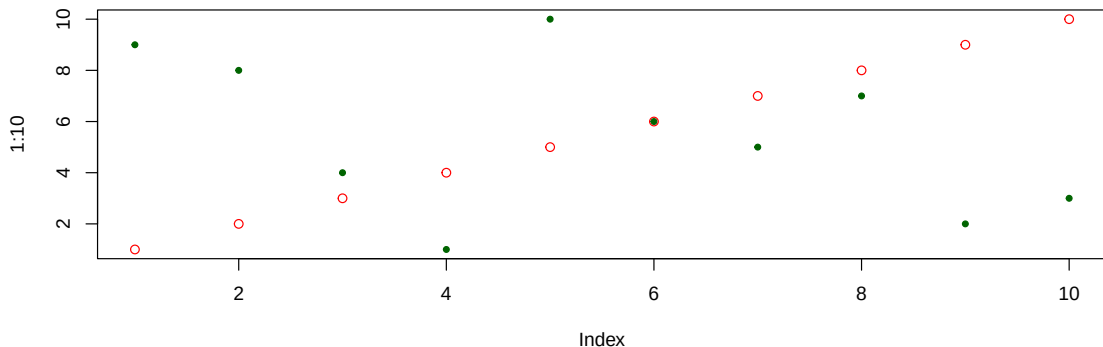
```
library(tidyverse)
ggplot(data.frame(p = c(0:25, 32:127))) +
  scale_y_continuous(name = "") + scale_x_continuous(name = "") +
  scale_shape_identity() +
  geom_point(aes(x = p %% 16, y = p %% 16, shape = p), size = 5, fill = "red") +
  geom_text(aes(x = p %% 16, y = p %% 16 + 0.4, label = p), size = 3)
```



High-level vs. Low-level Plots

- **High-level:** creates a new plot (plot, hist, boxplot, ...)
- **Low-level:** adds to existing plot (points, lines, abline, legend, ...)

```
plot(1:10, col = "red")           # high-level
points(sample(1:10, 10), col = "darkgreen", pch = 20) # low-level
```



Graphics Parameters: `par()`

```
par(c("mar", "bg")) # show specific parameters
```

```
$mar  
[1] 5.1 4.1 4.1 2.1
```

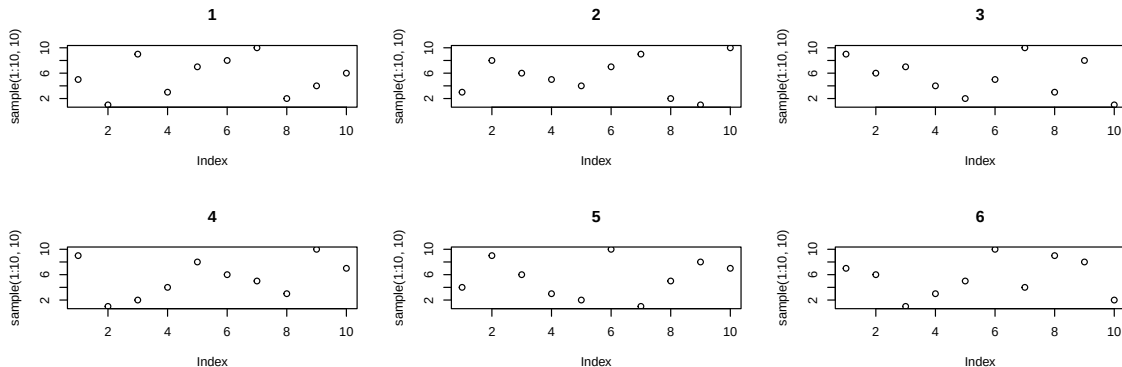
```
$bg  
[1] "transparent"
```

Always backup before changing `par()`:

```
oldpar <- par()  
# ... make changes ...  
par(oldpar) # restore
```

Multi-panel Layout with par()

```
par(mfrow = c(2, 3)) # 2 rows, 3 cols, fill by row
for (i in 1:6)
  plot(sample(1:10, 10), main = i)
```



`mfcol = c(2,3)` fills by column instead.

Graphics Devices

The **graphics device** is the output destination:

- **Screen:** `X11()` (Linux), `windows()` (Windows), `quartz()` (macOS)
- **File:** `pdf()`, `png()`, `jpeg()`

```
pdf(file = "myplot.pdf", height = 5, width = 5)
plot(1:10)
dev.off() # MUST close the device!
```

Always use PDF for bioinformatics figures — it's a vector format (infinite zoom, editable in Illustrator).

Section 2

Introduction to ggplot2

Grammar of Graphics Philosophy

ggplot2 is based on **The Grammar of Graphics** (Leland Wilkinson):

- **Data** — the dataset
- **Aesthetics (aes)** — mapping variables to visual properties
- **Geometries (geom)** — visual marks (points, lines, bars, ...)
- **Scales** — control how data values map to visual values
- **Facets** — small multiples
- **Themes** — non-data ink

A Simple ggplot2 Example

```
library(tidyverse)

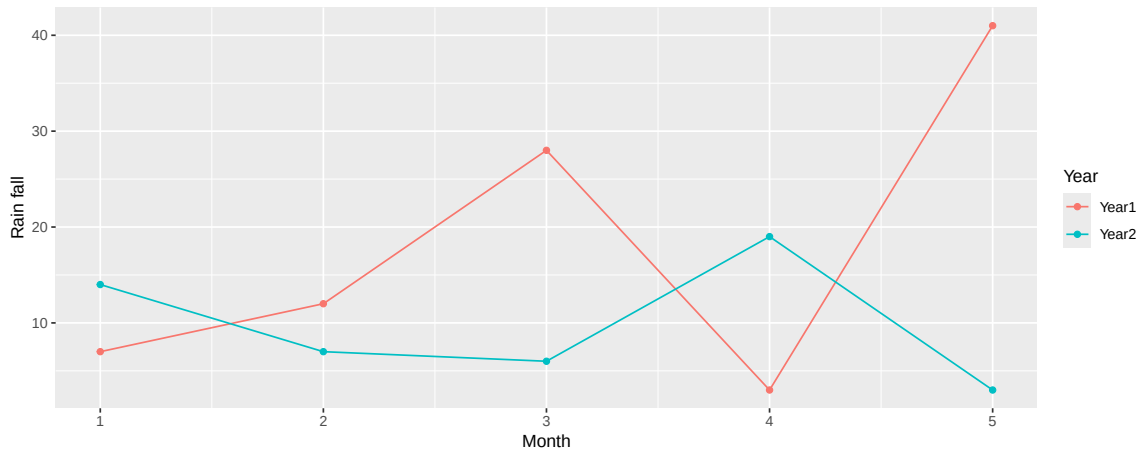
year1 <- c(7, 12, 28, 3, 41)
year2 <- c(14, 7, 6, 19, 3)

df <- rbind(
  tibble(month = 1:length(year1), value = year1, cat = "Year1"),
  tibble(month = 1:length(year2), value = year2, cat = "Year2"))

plot1 <- ggplot(df, aes(x = month, y = value, colour = cat)) +
  geom_line() + geom_point() +
  xlab("Month") + ylab("Rain fall") +
  labs(colour = "Year")
```

The Plot

```
plot1
```

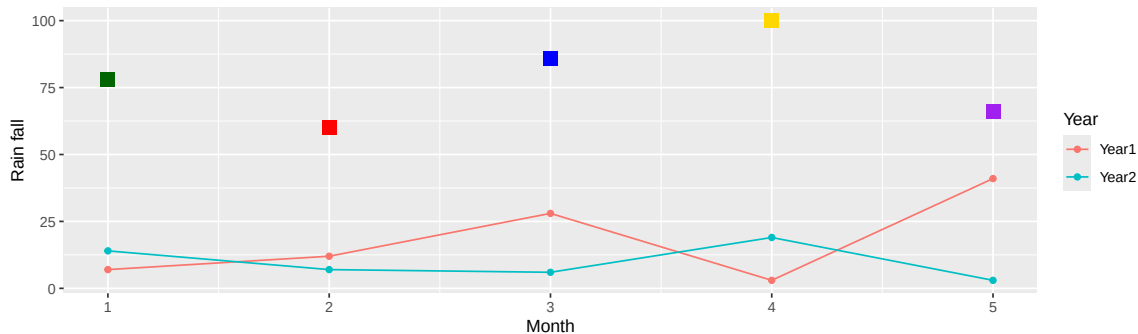


ggplot2 Anatomy Explained

- **aes()** (aesthetics): maps data to visual properties — x, y, colour, fill, shape, size, group
- **Layers** (`geom_XXX`): each plot can have multiple layers; layers can inherit or override aes and data
- **Other controls**: scales, facets, themes, labels

Layers Can Use Their Own Data

```
plot1 +
  geom_point(data = data.frame(x2 = 1:5, y2 = sample(30:100, 5)),
    aes(x = x2, y = y2),
    colour = c("darkgreen", "red", "blue", "gold", "purple"),
    shape = 15, size = 4)
```



Layers Can Use Their Own Data, cont.

Key points:

- ① Axes auto-adjust to data
- ② Plots can be stored in variables and incrementally built
- ③ Layer-specific data needs `data = argument`

ggplot2: Pros & Cons

Pros:

- Powerful, professional
- Complex yet beautiful
- Canvas auto-adjusts — you focus on the data story

Cons:

- Steep learning curve
- Many parameters to remember

The Four Core Components of ggplot2

- ① **Layers** — `geom_xxx`
- ② **Scales** — `scale_xxx_yyy`
- ③ **Coordinate systems** — `coord_xxx`
- ④ **Facets** — `facet_xxx`

Section 3

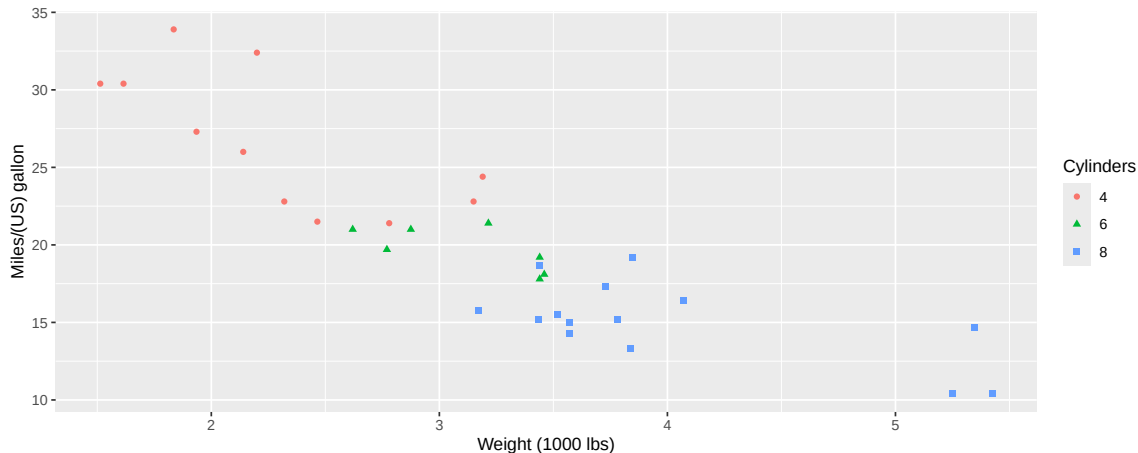
1. Layers (geoms)

Common Geoms

geom	Use case
<code>geom_point()</code>	Scatter plots
<code>geom_line()</code>	Line charts
<code>geom_smooth()</code>	Trend lines
<code>geom_bar()</code> / <code>geom_col()</code>	Bar charts
<code>geom_boxplot()</code>	Box plots
<code>geom_violin()</code>	Violin plots
<code>geom_histogram()</code>	Histograms
<code>geom_density()</code>	Density plots
<code>geom_path()</code>	Paths

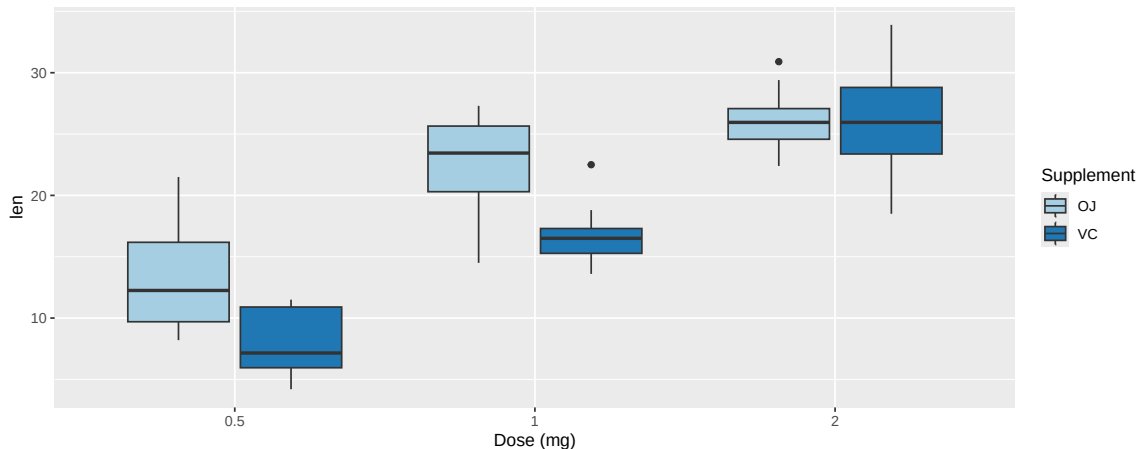
Scatter Plot with ggplot2

```
ggplot(mtcars, aes(x = wt, y = mpg, colour = factor(cyl), shape = factor(cyl))) +
  geom_point() +
  xlab("Weight (1000 lbs)") + ylab("Miles/(US) gallon") +
  labs(colour = "Cylinders", shape = "Cylinders")
```



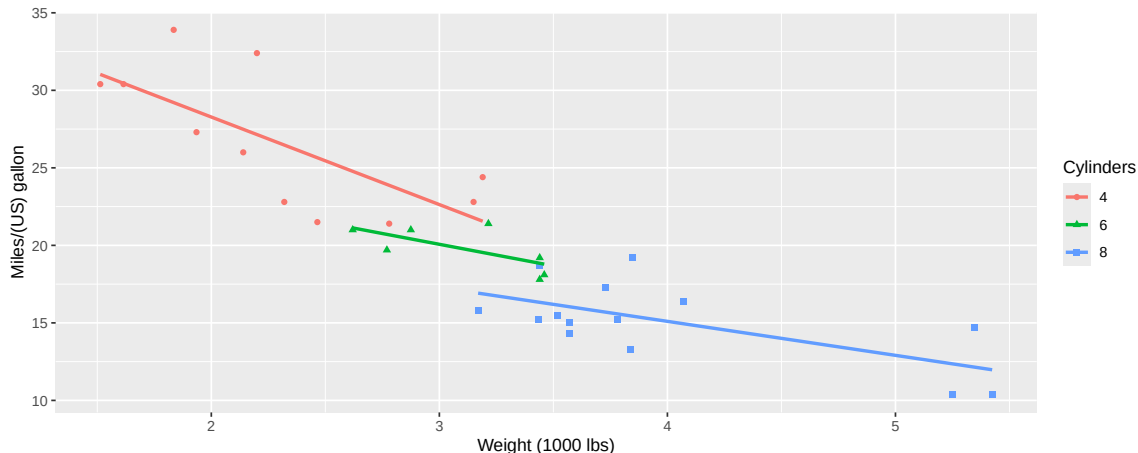
Box Plot

```
ggplot(ToothGrowth, aes(x = factor(dose), y = len, fill = supp)) +  
  geom_boxplot() + scale_fill_brewer(palette = "Paired") +  
  labs(fill = "Supplement", x = "Dose (mg)")
```



Multiple Geoms in One Plot

```
ggplot(mtcars, aes(x = wt, y = mpg, colour = factor(cyl), shape = factor(cyl))) +
  geom_point() + ## layer 1
  geom_smooth(method = "lm", se = FALSE) + ## layer 2; both layers use the same data
  xlab("Weight (1000 lbs)") + ylab("Miles/(US) gallon") +
  labs(colour = "Cylinders", shape = "Cylinders")
```



Section 4

2. Scales

Scale Rule

Scale is a function that maps data values to visual values.

`scale_<aesthetic>_<method>`, e.g. `scale_color_manual()`

Four aesthetic types:

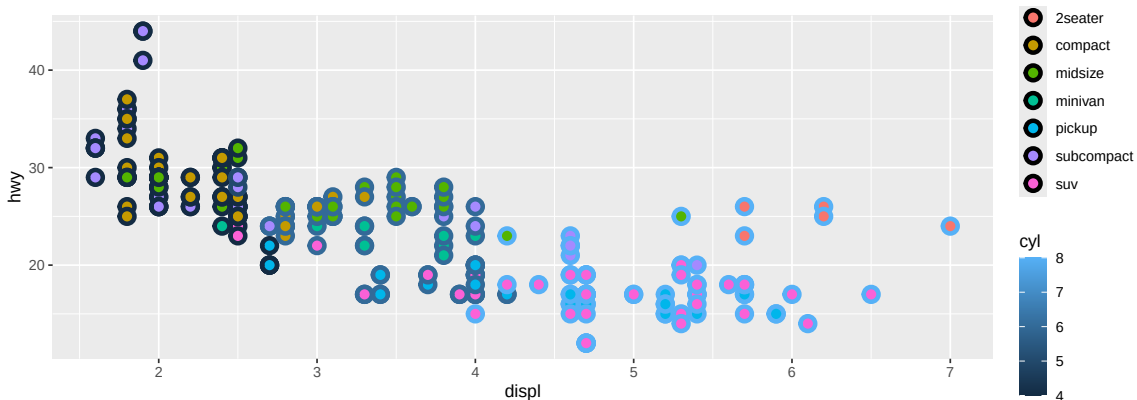
- `scale_color_...` — outline/stroke color
- `scale_fill_...` — fill color
- `scale_shape_...` — point shape
- `scale_size_...` — point/text size

colour vs. fill

- **colour**: outline (stroke) of a geom
- **fill**: interior of a geom

Point shapes **21–25** have **both** colour and fill.

```
mpg %>% ggplot(aes(displ, hwy)) +  
  geom_point(aes(colour = cyl, fill = class), shape = 21, size = 3, stroke = 2)
```



How the Fill Color is Determined?

- `fill = class` means the fill color is determined by the `class` variable.
- `class` **contains categorical values** and has **7 levels**, so the fill color is determined by the default discrete color palette.

Here is how the discrete color palette looks like:

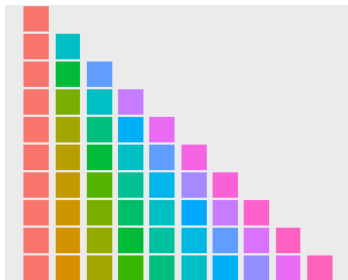
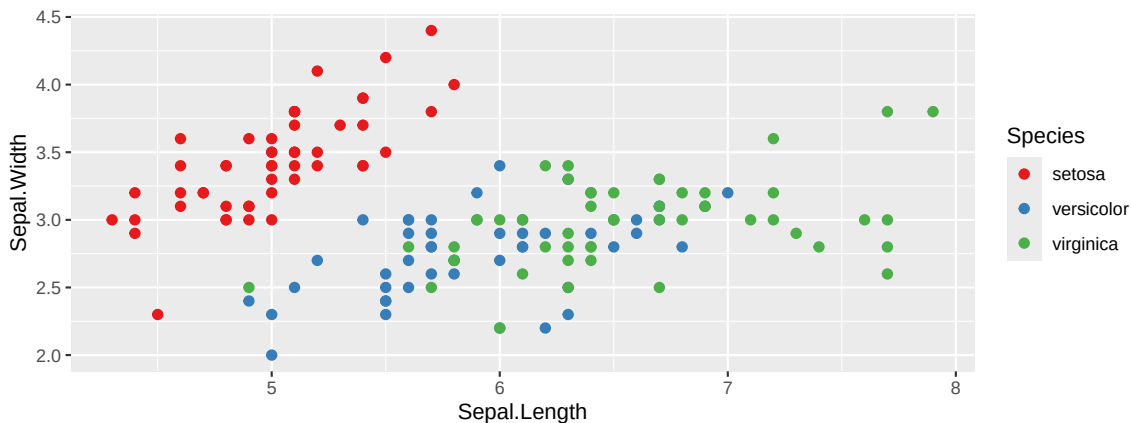


Figure 1: the 7th row of this palette will be chosen!

Discrete Color Palettes: `scale_colour_brewer()`

```
ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +  
  geom_point(size = 2) +  
  scale_color_brewer(palette = "Set1")
```



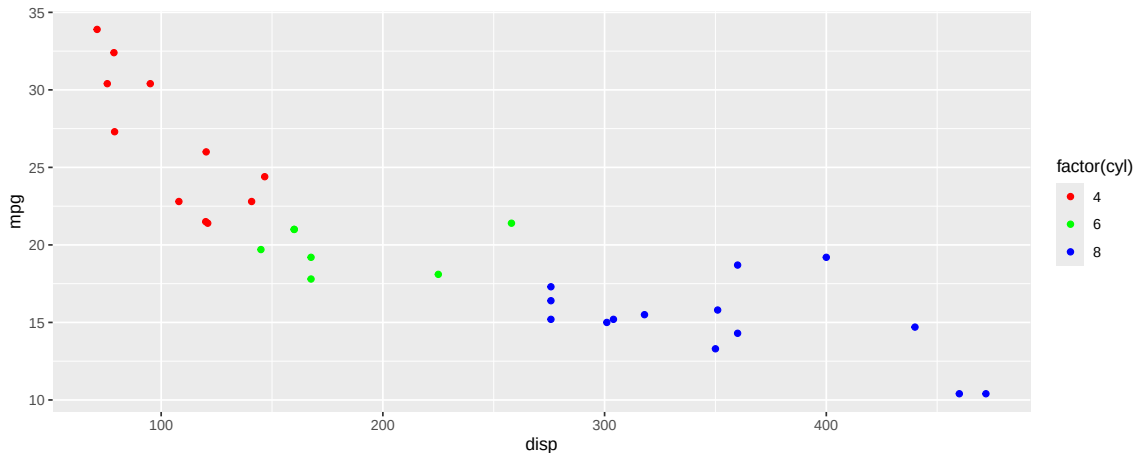
Discrete Color Palettes: `scale_colour_brewer()`, cont.

Three palette types:

- **Sequential:** Blues, Greens, Reds, ...
- **Qualitative:** Set1, Set2, Set3, Dark2, Paired, ...
- **Diverging:** RdBu, RdYlGn, Spectral, ...

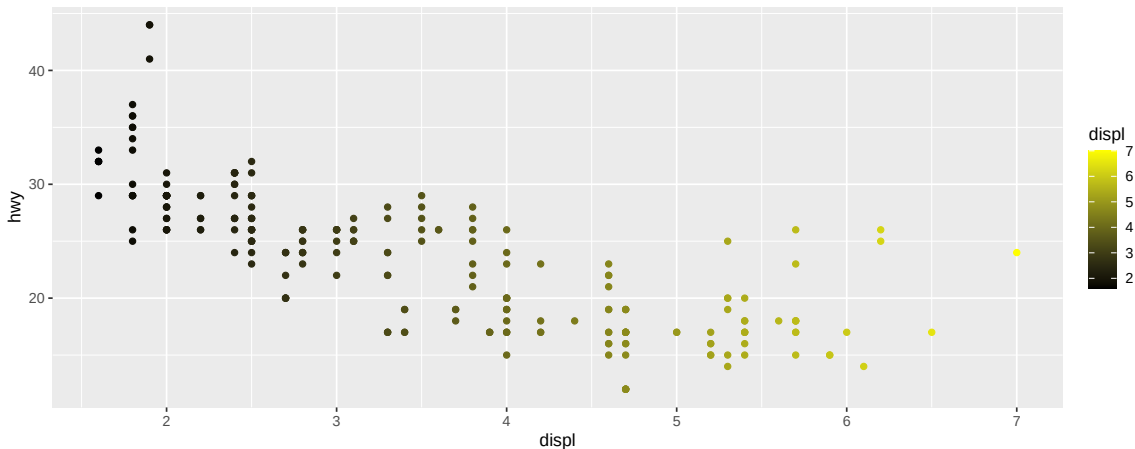
Manual Colors: `scale_colour_manual()`

```
mtcars %>% ggplot(aes(displ, mpg)) +  
  geom_point(aes(color = factor(cyl))) +  
  scale_color_manual(breaks = c("4", "6", "8"),  
                    values = c("red", "green", "blue"))
```



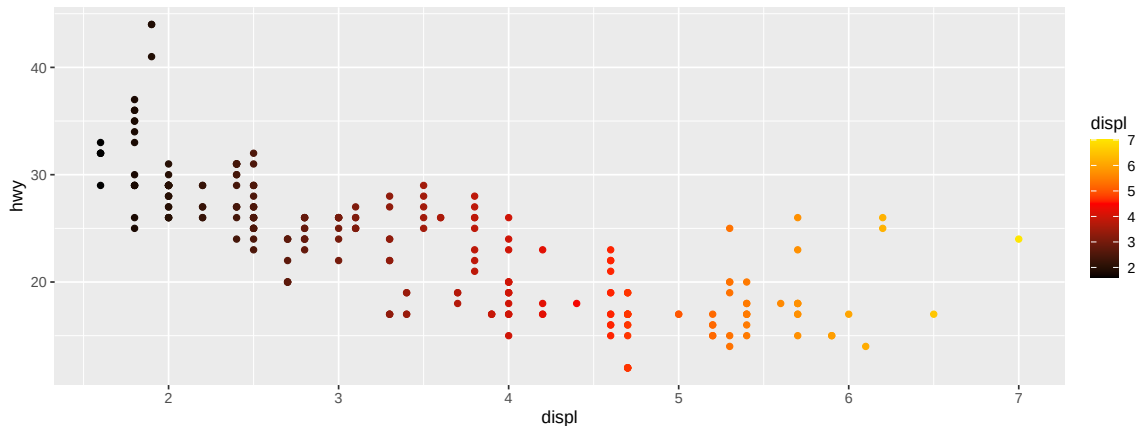
Gradient Colors for Continuous Variables

```
mpg %>% ggplot(aes(displ, hwy)) +  
  geom_point(aes(color = displ)) +  
  scale_color_gradient(low = "black", high = "yellow")
```



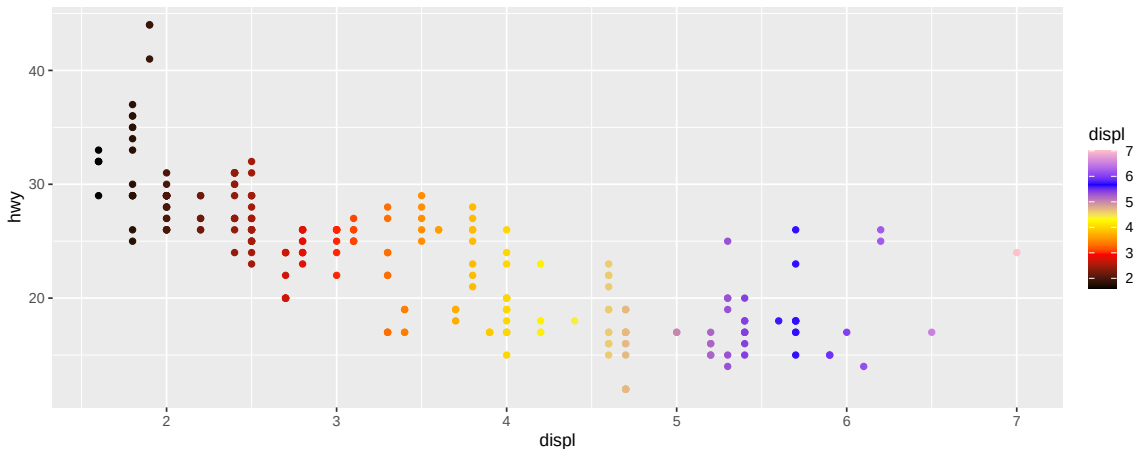
Two-Color Gradient: `scale_colour_gradient2()`

```
mpg %>% ggplot(aes(displ, hwy)) +  
  geom_point(aes(color = displ)) +  
  scale_colour_gradient2(low = "black", mid = "red", high = "yellow", midpoint = 4.5)
```



N-Color Gradient: `scale_colour_gradientn()`

```
mpg %>% ggplot(aes(displ, hwy)) +  
  geom_point(aes(color = displ)) +  
  scale_colour_gradientn(colors = c("black", "red", "yellow", "blue", "pink"))
```



Publication-Ready Palettes: ggsci

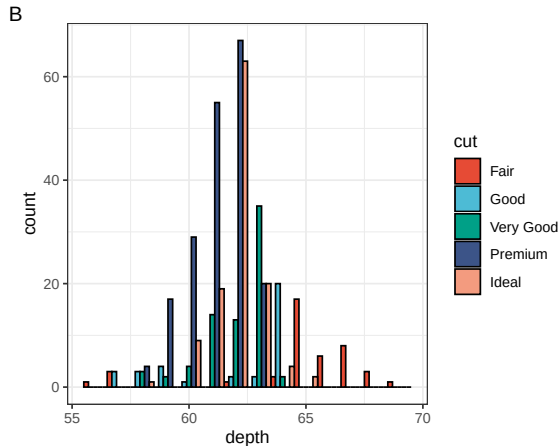
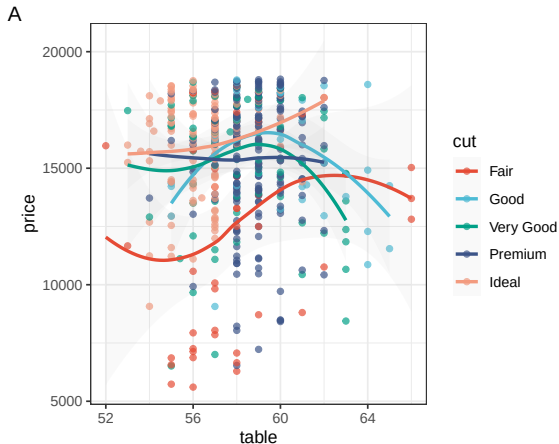
```
library(ggsci)
library(gridExtra)

data("diamonds")
p1 <- ggplot(subset(diamonds, carat >= 2.2),
             aes(x = table, y = price, colour = cut)) +
  geom_point(alpha = 0.7) +
  geom_smooth(method = "loess", alpha = 0.05, size = 1, span = 1) +
  theme_bw() + labs(tag = "A")

p2 <- ggplot(subset(diamonds, carat > 2.2 & depth > 55 & depth < 70),
             aes(x = depth, fill = cut)) +
  geom_histogram(colour = "black", binwidth = 1, position = "dodge") +
  theme_bw() + labs(tag = "B")
```

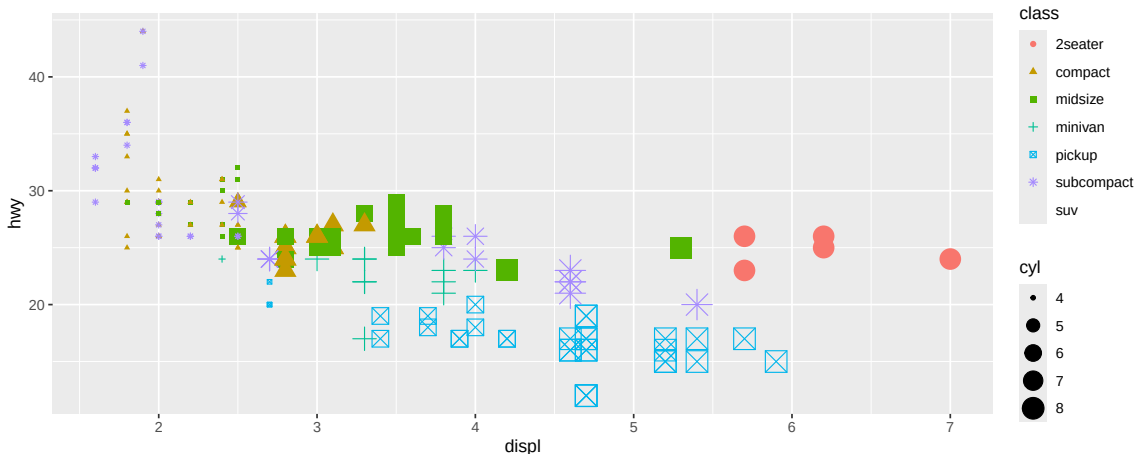
ggsci — Nature Publishing Group Style

```
p1_npg <- p1 + scale_color_npg()
p2_npg <- p2 + scale_fill_npg()
grid.arrange(p1_npg, p2_npg, ncol = 2)
```



Size and Shape Scales

```
mpg %>% ggplot(aes(displ, hwy)) +  
  geom_point(aes(size = cyl, colour = class, shape = class))
```



aes() Inside vs. Outside

Inside aes(): maps to **data column** — varies by data values

Outside aes(): uses **fixed value** — constant for all points

```
# Inside — varies by data
```

```
geom_point(aes(size = cyl, colour = class))
```

```
# Outside — fixed value
```

```
geom_point(size = 2, colour = "green", shape = 21)
```

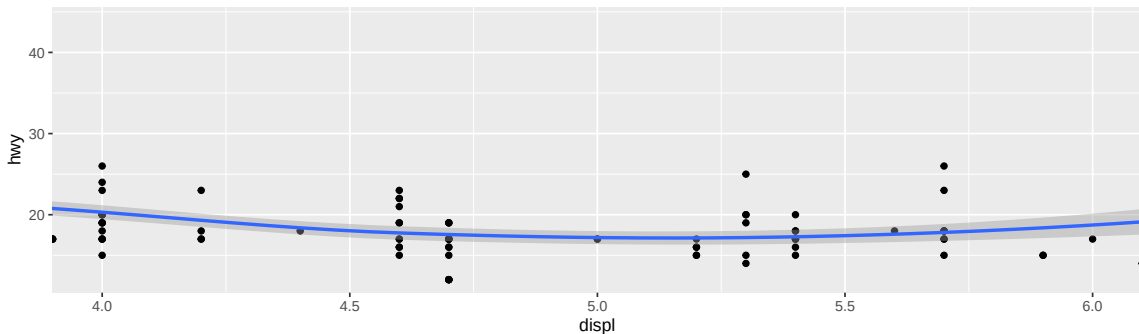
Section 5

3. Coordinate Systems

Linear Coordinate Systems

```
p1 <- ggplot(mpg, aes(displ, hwy)) + geom_point() + geom_smooth()

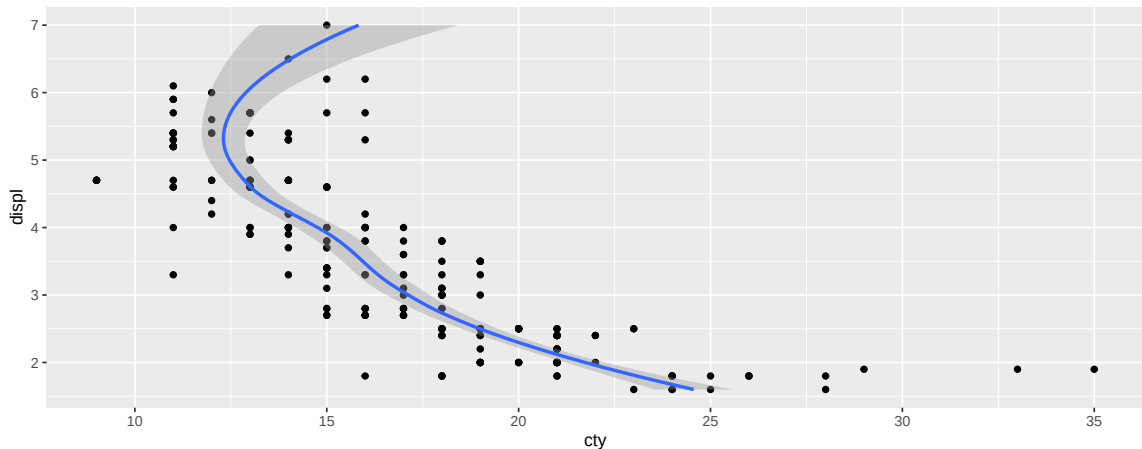
p1 + coord_cartesian(xlim = c(4, 6))  # zoom in
```



Cartesian coordinate system is the default in R.

coord_flip()

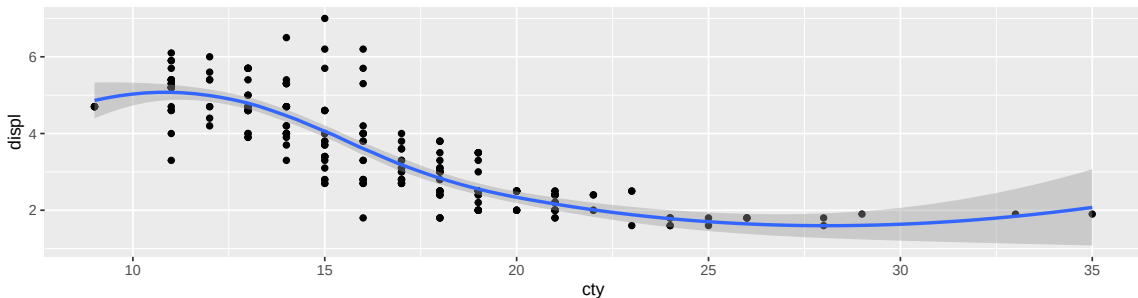
```
# Flip x and y axes
ggplot(mpg, aes(displ, cty)) + geom_point() + geom_smooth() + coord_flip()
```



coord_fixed()

`coord_fixed(ratio = 1)` — unit on x equals unit on y (e.g. for maps).

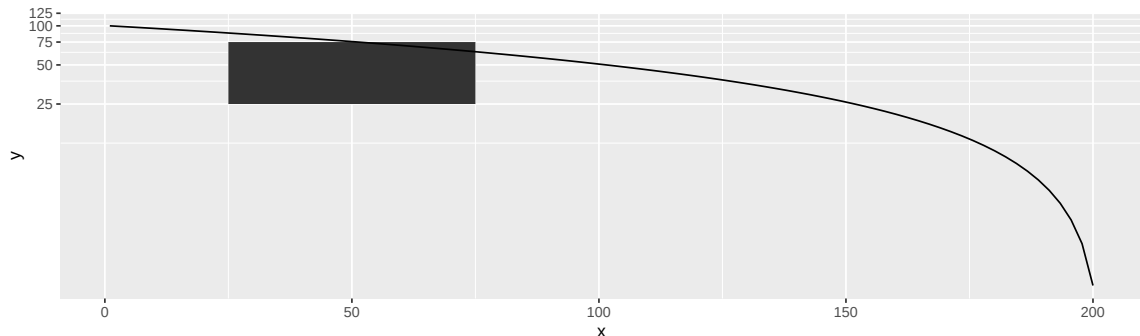
```
ggplot(mpg, aes(cty, displ)) + geom_point() + geom_smooth() + coord_fixed()
```



coord_trans() — Nonlinear Transformations

```
rect <- data.frame(x = 50, y = 50)
line <- data.frame(x = c(1, 200), y = c(100, 1))
base <- ggplot(mapping = aes(x, y)) +
  geom_tile(data = rect, aes(width = 50, height = 50)) +
  geom_line(data = line)

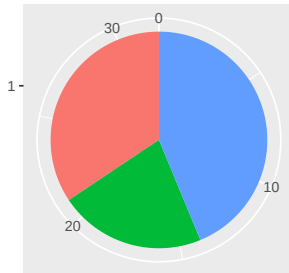
base + coord_trans(y = "log10")
```



coord_polar() — Pie Charts

```
base2 <- ggplot(mtcars, aes(factor(1), fill = factor(cyl))) +
  geom_bar(width = 1) + theme(legend.position = "none") +
  scale_x_discrete(NULL, expand = c(0, 0)) +
  scale_y_continuous(NULL, expand = c(0, 0))

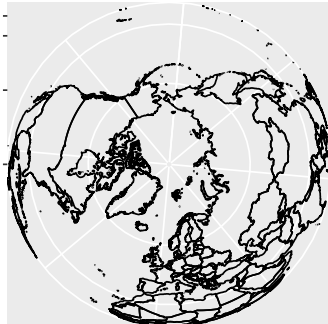
base2 + coord_polar(theta = "y") # bar → pie
```



coord_map() — World Maps

```
library(mapproj)
world <- map_data("world")
worldmap <- ggplot(world, aes(long, lat, group = group)) +
  geom_path() +
  scale_y_continuous(NULL, breaks = (-2:3) * 30, labels = NULL) +
  scale_x_continuous(NULL, breaks = (-4:4) * 45, labels = NULL)

worldmap + coord_map("ortho") # orthographic projection
```

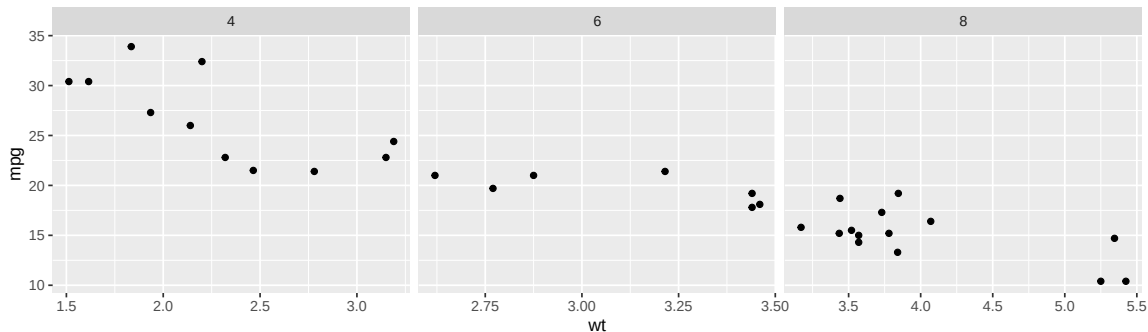


Section 6

4. Facets

facet_grid()

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point() +  
  facet_grid(. ~ cyl, scales = "free")
```

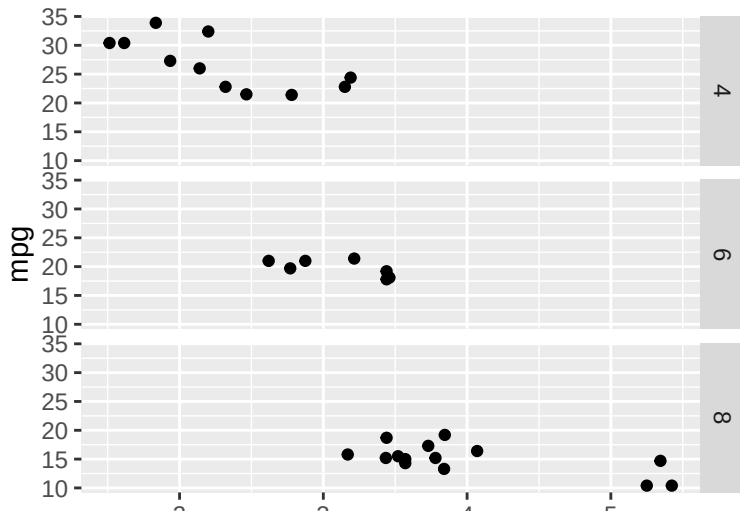


`facet_grid(rows ~ cols)` — . means no faceting on that dimension.



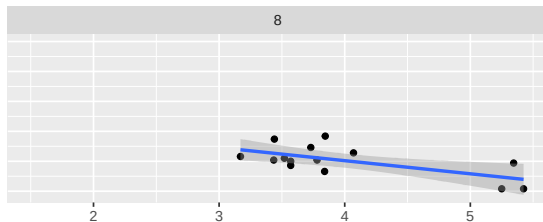
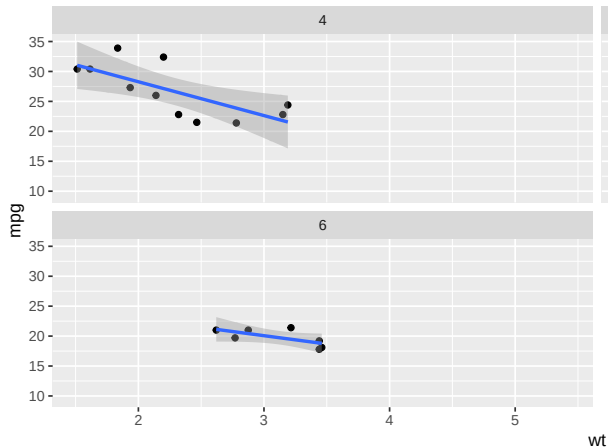
facet_grid() by Row

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point() +  
  facet_grid(cyl ~ .)
```



facet_wrap()

```
ggplot(mtcars, aes(x = wt, y = mpg)) + geom_point() + geom_smooth(method = "lm") +  
  facet_wrap(. ~ cyl, ncol = 2, dir = "v")
```

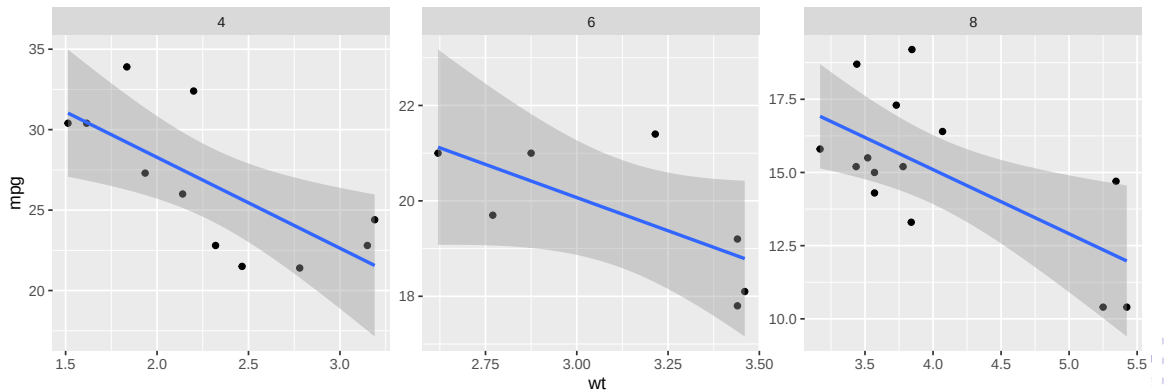


Key parameters: `nrow`, `ncol`, `scales`, `dir`

Key parameters - scales

Here is an example that each facet has its own scale:

```
ggplot(mtcars, aes(x = wt, y = mpg)) + geom_point() + geom_smooth(method = "lm") +  
  facet_wrap(. ~ cyl, scales = "free")
```



Section 7

Summary

What We Covered

Component	Key Functions
Base R Graphics	<code>plot()</code> , <code>par()</code> , <code>pdf()</code> , <code>dev.off()</code>
ggplot2 Basics	<code>ggplot()</code> , <code>aes()</code> , geom layers
Geoms	<code>geom_point</code> , <code>geom_line</code> , <code>geom_bar</code> , <code>geom_boxplot</code> , <code>geom_histogram</code> , <code>geom_density</code> , <code>geom_smooth</code>
Scales	<code>scale_color_brewer</code> , <code>scale_color_manual</code> , <code>scale_color_gradient*</code> , <code>ggsci</code>
Coordinates	<code>coord_cartesian</code> , <code>coord_flip</code> , <code>coord_fixed</code> , <code>coord_trans</code> , <code>coord_polar</code> , <code>coord_map</code>
Facets	<code>facet_grid</code> , <code>facet_wrap</code>



Key Takeaway

ggplot2 = **grammar of graphics** — build plots layer by layer, with consistent logic.

Next Talk

Talk 06: Data Visualization II — Advanced ggplot2, Themes & Saving Plots

Homework

Complete Homework_AI_powered_R/talk05-homework.qmd